

pandas 데이터 파악과 조작

분석할 데이터를 수집(확보)하면 데이터의 특징을 파악하고 다루기 쉽게 변형하는 작업을 수행해야 한다

#2. 데이터 조작(가공)

- 데이터 개수 세기 : `count()`, `value_counts()`
- 데이터 정렬 : `sort_values()`, `sort_index()`
- 데이터 순위 : `rank()`
- 데이터 집계 : 합계(`sum()`), 평균(`mean()`), 최대(`max()`), 최소(`min()`) 등
- 데이터 삭제 : `drop(axis=0/1)`
- 결측치 처리 : `dropna(axis=0/1, subset, inplace)`
- 데이터 변경 :
 - 자료형 변경 : `astype()`
 - 수치형 데이터를 범주형 데이터로 변경 :
 - 구간을 지정하여 범주화 : `cut(data, bins, labels)`
 - 동일한 개수를 갖도록 범주화 : `qcut(data, bins_num, labels)`
- 행/열에 동일한 함수 적용 : `apply()`, `applymap()`

```
In [1]: import numpy as np
import pandas as pd
print(f'Numpy:{np.__version__}, Pandas:{pd.__version__}')
```

Numpy:2.5.2, Pandas:3.0.5

```
In [2]: from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
```

1. 데이터 개수 세기(count)

: 데이터 빈도(frequency) 계산

- **count() 함수**
 - 시리즈의 경우 빈도 계산
 - 데이터프레임의 경우 각 컬럼 또는 행의 데이터 빈도 계산
 - NaN값은 세지 않음
- **value_counts() 함수**
 - 이산형, 범주형 데이터의 범주별 빈도 계산

1) count() 적용하여 개수 세기

- 형식 : **Series.count(level=None)**
- 형식 : **DataFrame.count(axis=0, level=None, numeric_only=False)**
 - axis : 0 or 'index', 1 or 'columns' , default 0
 - level : int or str, optional (multiIndex의 경우)

- numerical_only : float, int, boolean data만 포함

① Series 데이터에 count() 함수 적용

```
In [3]: df_tit = pd.read_csv('data/train.csv')
df_tit.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    str
4   Sex          891 non-null    str
5   Age         714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    str
9   Fare         891 non-null    float64
10  Cabin        204 non-null    str
11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
```

```
In [4]: df_tit.columns
```

```
Out[4]: Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
              'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
              dtype='str')
```

```
In [5]: df_tit['Survived'].count()
```

```
Out[5]: np.int64(891)
```

```
In [6]: df_tit.Pclass.count()
```

```
Out[6]: np.int64(891)
```

```
In [7]: for col in df_tit.columns:
        print(f'{col}: {df_tit[col].count()}')
```

```
PassengerId: 891
Survived: 891
Pclass: 891
Name: 891
Sex: 891
Age: 714
SibSp: 891
Parch: 891
Ticket: 891
Fare: 891
Cabin: 204
Embarked: 889
```

② 데이터 프레임에 count() 함수 적용

- 각 열마다 데이터 개수를 세기 때문에 누락된 부분을 찾을 때 유용

```
In [8]: df_tit.count()
```

```
Out[8]: PassengerId    891
Survived      891
Pclass        891
Name          891
Sex           891
Age          714
SibSp         891
Parch         891
Ticket        891
Fare          891
Cabin         204
Embarked      889
dtype: int64
```

2) value_counts()로 범주별 빈도 계산

- 문자열 값을 갖는 데이터의 경우 **범주별 빈도(비율) 계산**
 - ex. 성별(남,여), 선호도(좋다, 보통, 싫다), 혈액형(A,B,O,AB) 등
- 수치형 데이터의 값별 빈도 계산
- 형식 : **Series.value_counts(normalize=False, sort=True, ascending=False, bins=None, dropna=True)**
 - normalize : True인 경우 상대빈도 계산
 - sort : 빈도 크기별로 정렬
 - ascending : 오름차순으로 정렬
 - bins : 범주 구간 수 지정
 - dropna : False로 지정되면 NaN를 포함하여 빈도 계산
- 형식 : **DataFrame.value_counts(subset=None, normalize=False, sort=True, ascending=False, dropna=True)**

① 시리즈 데이터에 value_counts() 적용

- 시리즈의 값이 정수, 문자열인 경우 각각의 값이 나온 횟수를 셀 수 있음
- 파라미터 normalize=True 를 사용하면 각 값 및 범주형 데이터의 비율(상대빈도)을 계산
 - 시리즈.value_counts(normalize=True)

```
In [9]: df_tit.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    str
4   Sex          891 non-null    str
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    str
9   Fare         891 non-null    float64
10  Cabin        204 non-null    str
11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
```

- 정수형 값을 갖는 시리즈 데이터에 value_counts() 적용

```
In [10]: df_tit['Survived'].value_counts()
```

```
Out[10]: Survived
0      549
1      342
Name: count, dtype: int64
```

```
In [11]: df_tit.Pclass.value_counts()
```

```
Out[11]: Pclass
3      491
1      216
2      184
Name: count, dtype: int64
```

```
In [12]: df_tit.Pclass.value_counts(normalize=True)
```

```
Out[12]: Pclass
3      0.551066
1      0.242424
2      0.206510
Name: proportion, dtype: float64
```

```
In [13]: df_tit.Pclass.value_counts(ascending=True)
```

```
Out[13]: Pclass
2      184
1      216
3      491
Name: count, dtype: int64
```

```
In [14]: df_tit.Pclass.value_counts(ascending=False)
```

```
Out[14]: Pclass
3      491
1      216
2      184
Name: count, dtype: int64
```

- 실수형 값을 갖는 시리즈 데이터에 value_counts() 적용

```
In [15]: df_tit.Age.value_counts(bins=6)
```

```
Out[15]: (13.683, 26.947]    248
(26.947, 40.21]         245
(40.21, 53.473]        100
(0.339, 13.683]         71
(53.473, 66.737]        43
(66.737, 80.0]          7
Name: count, dtype: int64
```

```
In [16]: df_tit.Age.value_counts(bins=6, sort=False)
```

```
Out[16]: (0.339, 13.683]        71
(13.683, 26.947]        248
(26.947, 40.21]         245
(40.21, 53.473]         100
(53.473, 66.737]         43
(66.737, 80.0]           7
Name: count, dtype: int64
```

- 문자열 값을 갖는 시리즈 데이터에 value_counts() 적용

```
In [17]: df_tit.Embarked.value_counts(sort=False)
```

```
Out[17]: Embarked
S      644
C      168
Q       77
Name: count, dtype: int64
```

```
In [18]: df_tit.Sex.value_counts()
```

```
Out[18]: Sex
male      577
female    314
Name: count, dtype: int64
```

```
In [19]: df_tit.Sex.value_counts(sort=False)
```

```
Out[19]: Sex
male      577
female    314
Name: count, dtype: int64
```

```
In [20]: df_tit.Sex.value_counts(normalize=True)*100
```

```
Out[20]: Sex
male      64.758698
female    35.241302
Name: proportion, dtype: float64
```

② 데이터프레임에 value_counts() 함수 사용

- 행(row)을 하나의 value로 설정하고 동일한 행이 몇 번 나타났는지의 빈도를 Series로 반환

```
In [21]: df = pd.DataFrame({'num_legs': [2, 4, 4, 6, 2],
                           'num_wings': [2, 0, 0, 0, 2]},
                           index=['falcon', 'dog', 'cat', 'ant', 'duck'])
df
```

```
Out[21]:
```

	num_legs	num_wings
falcon	2	2
dog	4	0
cat	4	0
ant	6	0
duck	2	2

```
In [22]: df.value_counts()
```

```
Out[22]: num_legs  num_wings
2           2           2
4           0           2
6           0           1
Name: count, dtype: int64
```

문제. 타이타닉 데이터에서 Pclass에 따른 생존/사망 빈도 계산

```
In [23]: df_tit[['Pclass', 'Survived']].head()
```

```
Out[23]:
```

	Pclass	Survived
0	3	0
1	1	1
2	3	1
3	1	1
4	3	0

```
In [24]: df_tit[['Pclass', 'Survived']].value_counts()
```

```
Out[24]:
```

Pclass	Survived	
3	0	372
1	1	136
3	1	119
2	0	97
	1	87
1	0	80

Name: count, dtype: int64

```
In [25]: df_tit[['Pclass', 'Survived']].value_counts(sort=False)
```

```
Out[25]:
```

Pclass	Survived	
3	0	372
1	1	136
3	1	119
1	0	80
2	1	87
	0	97

Name: count, dtype: int64

```
In [26]: df_tit[['Pclass', 'Survived']].value_counts(sort=False, normalize=True)*100
```

```
Out[26]:
```

Pclass	Survived	
3	0	41.750842
1	1	15.263749
3	1	13.355780
1	0	8.978676
2	1	9.764310
	0	10.886644

Name: proportion, dtype: float64

예. 결측치가 있는 데이터프레임

```
In [27]: df_tit.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  -
 0   PassengerId   891 non-null    int64
 1   Survived      891 non-null    int64
 2   Pclass        891 non-null    int64
 3   Name          891 non-null    str
 4   Sex           891 non-null    str
 5   Age           714 non-null    float64
 6   SibSp         891 non-null    int64
 7   Parch         891 non-null    int64
 8   Ticket        891 non-null    str
 9   Fare          891 non-null    float64
10   Cabin         204 non-null    str
11   Embarked      889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB

```

```
In [28]: df_tit.Embarked.value_counts()
```

```

Out[28]: Embarked
S      644
C      168
Q       77
Name: count, dtype: int64

```

```
In [29]: df_tit.Embarked.value_counts(dropna=False)
```

```

Out[29]: Embarked
S      644
C      168
Q       77
NaN        2
Name: count, dtype: int64

```

```
In [30]: df
```

```

Out[30]:

```

	num_legs	num_wings
falcon	2	2
dog	4	0
cat	4	0
ant	6	0
duck	2	2

```
In [31]: df.loc['cat','num_wings'] = np.nan
df
```

```

Out[31]:

```

	num_legs	num_wings
falcon	2	2.0
dog	4	0.0
cat	4	NaN
ant	6	0.0
duck	2	2.0

```
In [32]: df.value_counts()
```

```
Out[32]: num_legs  num_wings
2           2.0         2
4           0.0         1
6           0.0         1
Name: count, dtype: int64
```

```
In [33]: df.value_counts(dropna=False)
```

```
Out[33]: num_legs  num_wings
2           2.0         2
4           0.0         1
           NaN         1
6           0.0         1
Name: count, dtype: int64
```

유일한 값 세기 : unique()

```
In [34]: df_tit.Embarked.value_counts(dropna=False)
```

```
Out[34]: Embarked
S         644
C         168
Q          77
NaN          2
Name: count, dtype: int64
```

```
In [35]: df_tit.Embarked.unique()
```

```
Out[35]: <StringArray>
['S', 'C', 'Q', nan]
Length: 4, dtype: str
```

2. 데이터 정렬

- 데이터 정렬을 위한 정렬 함수 사용
- **sort_index()** : 인덱스를 기준으로 정렬
- **sort_values()** : 데이터 값을 기준으로 정렬

1) 시리즈 정렬

```
In [36]: df_tit.Pclass.value_counts()
```

```
Out[36]: Pclass
3         491
1         216
2         184
Name: count, dtype: int64
```

```
In [37]: df_tit.Pclass.value_counts().sort_index()
```

```
Out[37]: Pclass
1         216
2         184
3         491
Name: count, dtype: int64
```

```
In [38]: df_tit.Pclass.value_counts().sort_values()
```



```
Out[38]: Pclass
2      184
1      216
3      491
Name: count, dtype: int64
```

```
In [39]: df_tit.Pclass.value_counts().sort_values(ascending=False)
```

```
Out[39]: Pclass
3      491
1      216
2      184
Name: count, dtype: int64
```

2) 데이터 프레임 정렬

- **DataFrame.sort_values(by, axis=0, ascending=True, inplace=False, kind='quicksort', na_position='last', ignore_index=False, key=None)**
 - 특정 열 값 기준 정렬
 - 정렬시 기준열을 주어야 함. by 인수 사용(생략 불가)
 - by = 기준열, by=[기준열1, 기준열2]
 - 오름차순/내림차순 : ascending = True/False (생략하면 오름차순)
- **DataFrame.sort_index(axis=0, level=None, ascending=True, inplace=False, kind='quicksort', na_position='last', sort_remaining=True, ignore_index=False, key=None)**
 - DataFrame의 INDEX 기준 정렬
 - 오름차순/내림차순 : ascending = True/False (생략하면 오름차순)

```
In [40]: df
```

```
Out[40]:
```

	num_legs	num_wings
falcon	2	2.0
dog	4	0.0
cat	4	NaN
ant	6	0.0
duck	2	2.0

- 데이터프레임의 값을 기준으로 정렬

```
In [41]: df.sort_values(by='num_legs')
```

```
Out[41]:
```

	num_legs	num_wings
falcon	2	2.0
duck	2	2.0
cat	4	NaN
dog	4	0.0
ant	6	0.0

```
In [42]: df.sort_values(by='num_legs', ascending=False)
```

Out[42]:

	num_legs	num_wings
ant	6	0.0
cat	4	NaN
dog	4	0.0
falcon	2	2.0
duck	2	2.0

- 결측치를 맨 앞으로 정렬 : na_position='first'

```
In [43]: df.sort_values(by='num_wings')
```

Out[43]:

	num_legs	num_wings
dog	4	0.0
ant	6	0.0
falcon	2	2.0
duck	2	2.0
cat	4	NaN

```
In [44]: df.sort_values(by='num_wings', na_position='first')
```

Out[44]:

	num_legs	num_wings
cat	4	NaN
dog	4	0.0
ant	6	0.0
falcon	2	2.0
duck	2	2.0

- 정렬 기준이 여러 가지인 경우

```
In [45]: df.sort_values(by=['num_wings', 'num_legs'])
```

Out[45]:

	num_legs	num_wings
dog	4	0.0
ant	6	0.0
falcon	2	2.0
duck	2	2.0
cat	4	NaN

- 데이터프레임의 index를 기준으로 정렬

```
In [46]: df.sort_index()
```

Out[46]:

	num_legs	num_wings
ant	6	0.0
cat	4	NaN
dog	4	0.0
duck	2	2.0
falcon	2	2.0

In [47]: `df.sort_index(ascending=False)`

Out[47]:

	num_legs	num_wings
falcon	2	2.0
duck	2	2.0
dog	4	0.0
cat	4	NaN
ant	6	0.0

- `sort_values()`의 key 파라미터

```
In [48]: df2 = pd.DataFrame(  
    {  
        "col1": ["A", "A", "B", np.nan, "D", "C"],  
        "col2": [2, 1, 9, 8, 7, 4],  
        "col3": [0, 1, 9, 4, 2, 3],  
        "col4": ["a", "B", "c", "D", "e", "F"],  
    }  
)  
df2
```

Out[48]:

	col1	col2	col3	col4
0	A	2	0	a
1	A	1	1	B
2	B	9	9	c
3	NaN	8	4	D
4	D	7	2	e
5	C	4	3	F

In [49]: `df2.sort_values(by='col2')`

```
Out[49]:
```

	col1	col2	col3	col4
1	A	1	1	B
0	A	2	0	a
5	C	4	3	F
4	D	7	2	e
3	NaN	8	4	D
2	B	9	9	c

```
In [50]: df2.sort_values(by='col1')
```

```
Out[50]:
```

	col1	col2	col3	col4
0	A	2	0	a
1	A	1	1	B
2	B	9	9	c
5	C	4	3	F
4	D	7	2	e
3	NaN	8	4	D

```
In [51]: df2.sort_values(by='col4') # 알파벳의 대문자-소문자 순으로 정렬
```

```
Out[51]:
```

	col1	col2	col3	col4
1	A	1	1	B
3	NaN	8	4	D
5	C	4	3	F
0	A	2	0	a
2	B	9	9	c
4	D	7	2	e

```
In [52]: df2.col4.str.lower()
```

```
Out[52]:
```

0	a
1	b
2	c
3	d
4	e
5	f

Name: col4, dtype: str

```
In [53]: df2.sort_values(by='col4', key=lambda col:col.str.lower())
```

Out[53]:

	col1	col2	col3	col4
0	A	2	0	a
1	A	1	1	B
2	B	9	9	c
3	NaN	8	4	D
4	D	7	2	e
5	C	4	3	F

3. 데이터 순위 : rank()

- 가장 낮은 값부터 유효한 데이터 개수까지 순서를 매김
- Series와 DataFrame의 rank 메서드는 동점인 항목에 대해서는 평균 순위를 매겨 책정함
- 함수형식:

```
Series.rank(axis=0, method='average', numeric_only=False, na_option='keep',  
ascending=True, pct=False)
```

```
DataFrame.rank(axis=0, method='average', numeric_only=False, na_option='keep',  
ascending=True, pct=False)
```

- <https://pandas.pydata.org/docs/reference/api/pandas.Series.rank.html>
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.rank.html>

- 순위의 동률을 처리하는 method |method|설명| |---|---| |'average'|기본값. 같은 값을 갖는 항목들의 평균순위값을 순위로 매김 예. 6번째 7번째가 같은 수인경우 6.5| |'min'|같은 값을 갖는 그룹을 낮은 순위로 매김| |'max'|같은 값을 갖는 그룹을 높은 순위로 매김| |'first'|데이터 내의 위치에 따라 순위를 매김| |'dense'|method='min'과 같지만 같은 그룹 내에서 모두 같은 순위를 적용|

시리즈 데이터 순위 매기기

```
In [54]: s = pd.Series([7, -5, 7, 4, 2, 0, 4])  
s
```

```
Out[54]: 0    7  
1   -5  
2    7  
3    4  
4    2  
5    0  
6    4  
dtype: int64
```

```
In [55]: s.rank()
```

```
Out[55]: 0    6.5  
1    1.0  
2    6.5  
3    4.5  
4    3.0  
5    2.0  
6    4.5  
dtype: float64
```

- 데이터 상에서 나타나는 순서에 따라 순위 매기기 : method='first'

```
In [56]: s.rank(method='first')
```

```
Out[56]: 0    6.0  
         1    1.0  
         2    7.0  
         3    4.0  
         4    3.0  
         5    2.0  
         6    5.0  
         dtype: float64
```

```
In [57]: s.rank(method='min')
```

```
Out[57]: 0    6.0  
         1    1.0  
         2    6.0  
         3    4.0  
         4    3.0  
         5    2.0  
         6    4.0  
         dtype: float64
```

```
In [58]: s.rank(method='max')
```

```
Out[58]: 0    7.0  
         1    1.0  
         2    7.0  
         3    5.0  
         4    3.0  
         5    2.0  
         6    5.0  
         dtype: float64
```

```
In [59]: s.rank(method='dense')
```

```
Out[59]: 0    5.0  
         1    1.0  
         2    5.0  
         3    4.0  
         4    3.0  
         5    2.0  
         6    4.0  
         dtype: float64
```

- 내림차순으로 순위 매기기 : ascending=False

```
In [60]: s.rank(method='first', ascending=False)
```

```
Out[60]: 0    1.0  
         1    7.0  
         2    2.0  
         3    3.0  
         4    5.0  
         5    6.0  
         6    4.0  
         dtype: float64
```

데이터프레임 데이터 순위 매기기

```
In [61]: df3 = pd.DataFrame({'b':[4.3, 7, -3, 2],  
                             'a':[0, 1, 0, 1],  
                             'c':[-2, 5, 8, -2.5]})  
  
df3
```

```
Out[61]:
```

	b	a	c
0	4.3	0	-2.0
1	7.0	1	5.0
2	-3.0	0	8.0
3	2.0	1	-2.5

- 행에 대해 순위 매기기

```
In [62]: df3.rank(axis=0)
```

```
Out[62]:
```

	b	a	c
0	3.0	1.5	2.0
1	4.0	3.5	3.0
2	1.0	1.5	4.0
3	2.0	3.5	1.0

```
In [63]: df3.rank(axis='index')
```

```
Out[63]:
```

	b	a	c
0	3.0	1.5	2.0
1	4.0	3.5	3.0
2	1.0	1.5	4.0
3	2.0	3.5	1.0

- 열에 대해 순위 매기기

```
In [64]: df3.rank(axis=1)
```

```
Out[64]:
```

	b	a	c
0	3.0	2.0	1.0
1	3.0	1.0	2.0
2	1.0	2.0	3.0
3	3.0	2.0	1.0

```
In [65]: df3.rank(axis='columns')
```

Out[65]:

	b	a	c
0	3.0	2.0	1.0
1	3.0	1.0	2.0
2	1.0	2.0	3.0
3	3.0	2.0	1.0

```
In [66]: df3.rank(axis=0, method='min')
```

Out[66]:

	b	a	c
0	3.0	1.0	2.0
1	4.0	3.0	3.0
2	1.0	1.0	4.0
3	2.0	3.0	1.0

연습문제

- 1. 타이타닉 데이터에서 승객의 성별(Sex) 인원수, 나이별(Age) 인원수, 선실별(Pclass) 인원수, 사망/생존(Survived)인원수를 구하시오.

```
In [67]: df_tit.head()
```

Out[67]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833	C85	S
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

```
In [68]: # 성별(Sex) 인원수
df_tit.Sex.value_counts()
```



```
Out[68]: Sex
male      577
female    314
Name: count, dtype: int64
```

```
In [69]: # 나이별(Age) 인원수
df_tit['Age'].value_counts(bins=8)
```

```
Out[69]: (20.315, 30.263]    230
(30.263, 40.21]         155
(10.368, 20.315]        115
(40.21, 50.158]          86
(0.339, 10.368]          64
(50.158, 60.105]         42
(60.105, 70.052]         17
(70.052, 80.0]           5
Name: count, dtype: int64
```

```
In [70]: # 선실별(Pclass) 인원수
df_tit.Pclass.value_counts()
```

```
Out[70]: Pclass
3      491
1      216
2      184
Name: count, dtype: int64
```

```
In [71]: # 사망/생존(Survived) 인원수
df_tit.Survived.value_counts()
```

```
Out[71]: Survived
0      549
1      342
Name: count, dtype: int64
```

2. 성별 인원수는 인덱스 기준으로 정렬하고, 나이별 인원수는 값 기준으로 정렬하시오.

```
In [72]: # 성별 인원수를 인덱스 기준으로 정렬
df_tit.Sex.value_counts().sort_index()
```

```
Out[72]: Sex
female    314
male      577
Name: count, dtype: int64
```

```
In [73]: # 나이별 인원수는 값 기준 정렬
df_tit['Age'].value_counts(bins=8).sort_values(ascending=False)
```

```
Out[73]: (20.315, 30.263]    230
(30.263, 40.21]         155
(10.368, 20.315]        115
(40.21, 50.158]          86
(0.339, 10.368]          64
(50.158, 60.105]         42
(60.105, 70.052]         17
(70.052, 80.0]           5
Name: count, dtype: int64
```

4. 데이터의 행/열의 집계 및 통계 관련 함수

: 데이터프레임에 적용되는 집계/통계 관련 함수

- 합계 : **sum**(axis=0 또는 1)
 - 각 열의 합계 계산 : axis=0(index) => 기본값
 - 각 행의 합계 계산 : axis=1(columns)
 - `DataFrame.sum(*, axis=0, skipna=True, numeric_only=False, min_count=0, **kwargs)`
- 최소값 : **min**(axis=0 또는 1)
- 최대값 : **max**(axis=0 또는 1)
- 중앙값 : **median**(axis=0 또는 1)
- 평균 : **mean**(axis=0 또는 1)
 - `DataFrame.mean(*, axis=0, skipna=True, numeric_only=False, **kwargs)`
- 표준편차 : **std**(axis=0 또는 1)
- 상관관계수 : **corr**(method='pearson')
 - `DataFrame.corr(method='pearson', min_periods=1, numeric_only=False)`

예제 데이터 프레임 생성

예제 DF 생성 # 4행 8열의 데이터프레임 작성, 난수를 발생시키고 # 0-9범위에서 매번 같은 난수 발생되어 반환되도록 설정

```
In [74]: np.random.seed(1)

df4 = pd.DataFrame(np.random.randint(10, size=(4,8)))
df4
```

```
Out[74]:
```

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1

행/열의 합계(sum)

- 시리즈.sum()
- 데이터프레임.sum()

행 기준(axis=0)의 합계 계산

```
In [75]: df4
df4.sum()
```

```
Out[75]:
```

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1

```
Out[75]: 0    24
         1    33
         2    25
         3    24
         4    15
         5    10
         6     5
         7    16
         dtype: int64
```

```
In [76]: df4.sum(axis='index')
```

```
Out[76]: 0    24
         1    33
         2    25
         3    24
         4    15
         5    10
         6     5
         7    16
         dtype: int64
```

```
In [77]: df4.sum(axis=0)
```

```
Out[77]: 0    24
         1    33
         2    25
         3    24
         4    15
         5    10
         6     5
         7    16
         dtype: int64
```

열 기준(axis=1)의 합계 계산

```
In [78]: df4.sum(axis=1)
```

```
Out[78]: 0    35
         1    34
         2    41
         3    42
         dtype: int64
```

```
In [79]: df4.sum(axis='columns')
```

```
Out[79]: 0    35
         1    34
         2    41
         3    42
         dtype: int64
```

통계관련 함수

- 최소값 : min()
- 최대값 : max()
- 중앙값 : median()
- 평균 : mean()
- 표준편차 : std()
- 상관계수 : corr()

```
In [80]: df4
df4.min()      # axis=0 또는 'index'
df4.min(axis=1) # axis='columns'
```

```
Out[80]:
```

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1

```
Out[80]:
```

0	4
1	7
2	2
3	4
4	0
5	0
6	0
7	1

dtype: int32

```
Out[80]:
```

0	0
1	2
2	0
3	0

dtype: int32

```
In [81]: df4
df4.max()      # axis=0 또는 'index'
df4.max(axis=1) # axis='columns'
```

```
Out[81]:
```

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1

```
Out[81]:
```

0	9
1	9
2	9
3	9
4	9
5	7
6	4
7	7

dtype: int32

```
Out[81]:
```

0	9
1	9
2	9
3	9

dtype: int32

```
In [82]: # std() : ddof 기본값은 1로 자유도 n-1 => 표본의 표준편차
df4
df4.std()      # axis=0 또는 'index' ,
df4.std(axis=1) # axis='columns'
```

```
Out[82]:
```

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1

```
Out[82]:
```

0	2.160247
1	0.957427
2	2.986079
3	2.160247
4	4.112988
5	3.109126
6	1.892969
7	2.943920

dtype: float64

```
Out[82]:
```

0	3.622844
1	2.434866
2	3.181981
3	3.955105

dtype: float64

```
In [83]: df4.var()           # axis=0 또는 'index' ,
df4.var(axis=1)           # axis='columns'
```

```
Out[83]:
```

0	4.666667
1	0.916667
2	8.916667
3	4.666667
4	16.916667
5	9.666667
6	3.583333
7	8.666667

dtype: float64

```
Out[83]:
```

0	13.125000
1	5.928571
2	10.125000
3	15.642857

dtype: float64

```
In [84]: df4
df4.median()           # axis=0 또는 'index' ,
df4.median(axis=1)     # axis='columns'
```

```
Out[84]:
```

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1

```
Out[84]: 0    5.5
          1    8.5
          2    7.0
          3    5.5
          4    3.0
          5    1.5
          6    0.5
          7    4.0
          dtype: float64
```

```
Out[84]: 0    5.0
          1    4.0
          2    6.5
          3    6.5
          dtype: float64
```

```
In [85]: # 상관계수행렬 : 두변수간의 상관계수
df4
df4.corr() # method = 'pearson' 피어슨상관계수, {'pearson', 'kendall', 'spearman'}
```

```
Out[85]:
```

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1

```
Out[85]:
```

	0	1	2	3	4	5	6	7
0	1.000000	0.805823	-0.103348	-0.357143	0.937903	-0.545921	-0.081514	-0.838628
1	0.805823	1.000000	-0.495519	-0.805823	0.782992	-0.727860	0.505781	-0.827837
2	-0.103348	-0.495519	1.000000	0.413394	-0.373184	-0.089759	-0.840328	0.606696
3	-0.357143	-0.805823	0.413394	1.000000	-0.262613	0.843696	-0.733625	0.366900
4	0.937903	0.782992	-0.373184	-0.262613	1.000000	-0.299765	0.053517	-0.963526
5	-0.545921	-0.727860	-0.089759	0.843696	-0.299765	1.000000	-0.254864	0.254925
6	-0.081514	0.505781	-0.840328	-0.733625	0.053517	-0.254864	1.000000	-0.299074
7	-0.838628	-0.827837	0.606696	0.366900	-0.963526	0.254925	-0.299074	1.000000

```
In [86]: # 공분산(covariance) : 두 변수의 상관성 -> 두 변수 데이터 스케일에 영향을 받음
df4.cov()
```

```
Out[86]:
```

	0	1	2	3	4	5	6	7
0	4.666667	1.666667	-0.666667	-1.666667	8.333333	-3.666667	-0.333333	-5.333333
1	1.666667	0.916667	-1.416667	-1.666667	3.083333	-2.166667	0.916667	-2.333333
2	-0.666667	-1.416667	8.916667	2.666667	-4.583333	-0.833333	-4.750000	5.333333
3	-1.666667	-1.666667	2.666667	4.666667	-2.333333	5.666667	-3.000000	2.333333
4	8.333333	3.083333	-4.583333	-2.333333	16.916667	-3.833333	0.416667	-11.666667
5	-3.666667	-2.166667	-0.833333	5.666667	-3.833333	9.666667	-1.500000	2.333333
6	-0.333333	0.916667	-4.750000	-3.000000	0.416667	-1.500000	3.583333	-1.666667
7	-5.333333	-2.333333	5.333333	2.333333	-11.666667	2.333333	-1.666667	8.666667

```
In [87]: df
```

```
Out[87]:
```

	num_legs	num_wings
falcon	2	2.0
dog	4	0.0
cat	4	NaN
ant	6	0.0
duck	2	2.0

```
In [88]: df.corr()
```

```
Out[88]:
```

	num_legs	num_wings
num_legs	1.000000	-0.904534
num_wings	-0.904534	1.000000

- `idxmin()`, `idxmax()` : 최소값/최대값을 가진 인덱스 반환

`DataFrame.idxmin(axis=0, skipna=True, numeric_only=False)`

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.idxmin.html>

```
In [89]: df4
df4.min()
df4.idxmin()
```

```
Out[89]:
```

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1

```
Out[89]: 0 4
1 7
2 2
3 4
4 0
5 0
6 0
7 1
dtype: int32
```

```
Out[89]: 0 2
1 2
2 1
3 1
4 0
5 0
6 2
7 3
dtype: int64
```

```
In [90]: df4
df4.max()
df4.idxmax()
```

Out[90]:

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1

```
Out[90]: 0 9
1 9
2 9
3 9
4 9
5 7
6 4
7 7
dtype: int32
```

```
Out[90]: 0 3
1 1
2 0
3 2
4 3
5 2
6 1
7 0
dtype: int64
```

```
In [91]: df4
df4.min(axis=1)
df4.idxmin(axis=1)
```

Out[91]:

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1


```
Out[91]: 0    0
          1    2
          2    0
          3    0
          dtype: int32
```

```
Out[91]: 0    4
          1    2
          2    6
          3    6
          dtype: int64
```

데이터프레임의 행/열 추가

- 열추가 : `df['새로운열이름'] = 데이터값 (또는 배열)`
- 행추가 : `df.loc['새로운행이름'] = 데이터값 (또는 배열)`

각 행의 합계를 새로운 열로 추가

```
In [92]: df4
```

```
Out[92]:
```

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1

```
In [93]: df4.sum(axis='columns')
df4['total'] = df4.sum(axis='columns')
df4
```

```
Out[93]: 0    35
          1    34
          2    41
          3    42
          dtype: int64
```

```
Out[93]:
```

	0	1	2	3	4	5	6	7	total
0	5	8	9	5	0	0	1	7	35
1	6	9	2	4	5	2	4	2	34
2	4	7	7	9	1	7	0	6	41
3	9	9	7	6	9	1	0	1	42

각 열의 합계를 새로운 행으로 추가

: loc 인덱서를 사용하여 새로운 행 추가

```
In [94]: df4.sum()
```

```
Out[94]: 0      24
          1      33
          2      25
          3      24
          4      15
          5      10
          6       5
          7      16
        total    152
        dtype: int64
```

```
In [95]: df4.loc['Sum'] = df4.sum()
df4
```

Out[95]:

	0	1	2	3	4	5	6	7	total
0	5	8	9	5	0	0	1	7	35
1	6	9	2	4	5	2	4	2	34
2	4	7	7	9	1	7	0	6	41
3	9	9	7	6	9	1	0	1	42
Sum	24	33	25	24	15	10	5	16	152

```
In [96]: del df4['total']
df4
```

Out[96]:

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1
Sum	24	33	25	24	15	10	5	16

```
In [97]: # 삭제 :drop()
df4.drop(index='Sum')
df4
```

Out[97]:

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1

```
Out[97]:
```

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1
Sum	24	33	25	24	15	10	5	16

```
In [98]: df4.drop(index='Sum', inplace=True)  
df4
```

```
Out[98]:
```

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1

```
In [99]: df4.drop(columns=7)
```

```
Out[99]:
```

	0	1	2	3	4	5	6
0	5	8	9	5	0	0	1
1	6	9	2	4	5	2	4
2	4	7	7	9	1	7	0
3	9	9	7	6	9	1	0

학습내용

1. 데이터 개수 세기

- count()
- value_counts()

2. 데이터 정렬(sort)

- sort_index()
- sort_values()

3. 데이터 순위(rank)

- Series.rank()
- DataFrame.rank()

4. 데이터 집계

- 행/열 집계 : `sum()`
 - 통계관련 함수 : `min()`, `max()`, `sum()`, `mean()`, `median()`, `std()`, `var()`, `idxmin()`, `idxmax()`, `corr()` 등
 - 데이터프레임 행/열 추가
-